Práctica de Aprendizaje por Refuerzo

Ginés Carrillo Ibáñez
gines.carrilloi@um.es

Yago Ibarrola Lapeña
yago.ibarrolal@um.es

Aarón Palomar Peña
aaron.p.p@um.es

14 de marzo de 2026

Resumen

En este documento se presentará uno de los trabajos que puede realizar como proyecto final y que deberán de ser expuestos en público.

Recuerde que la experimentación deberá de estar en un repositorio GitHub para poder contrastar lo que aquí explique. En el GitHub puede añadir tantos *notebooks* como considere. Pero aquí, el trabajo documental, ocupará 10 páginas descontadas la portada, el resumen, el índice y la bibliografía.

El contenido del repositorio y este documento deberán subirse al AV como evidencias del trabajo realizado.

Respete las dobles columnas. Si las suprime generará un documento que ocupará más páginas. Con las dobles columnas puede incluir más contenido.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.1.1. Descripción del problema y relevancia . . . 3

1.1.2. Motivación del trabajo . . . 3

1.1.3. Objetivos del informe . . . 3

1.1.4. Organización del documento . . . 3

1.2. Desarrollo . . . 3

1.2.1. Definición Formal del Problema . . . 3

1.2.2. Enfoques Existentes y Estado del Arte . . . 4

1.2.3. Contexto y Aplicaciones Específicas . . . 4

1.2.4. Métodos Utilizados . . . 4

1.3. Algoritmos . . . 5

1.3.1. Algoritmo ϵ-Greedy . . . 5

1.3.2. Algoritmo Softmax (Distribución de Gibbs) . . . 5

1.3.3. Algoritmos UCB (Upper Confidence Bound) . . . 5

1.4. Evaluación y Experimentos . . . 6

1.4.1. Configuración Experimental . . . 6

1.4.2. Resultados: Familia ϵ-Greedy . . . 6

1.4.3. Resultados: Familia Softmax . . . 7

1.4.4. Resultados: Familia UCB (Upper Confidence Bound) . . . 9

1.4.5. Análisis Comparativo Global . . . 10

1.5. Conclusiones . . . 11

2. Aprendizaje en Entornos Complejos: De Métodos Tabulares a Aproximaciones **12**

2.1. Introducción . . . 12

2.1.1. Descripción del problema y relevancia . . . 12

2.1.2. Motivación del trabajo . . . 12

2.1.3. Objetivos del capítulo . . . 12
2.1.4. Organización del capítulo . . . 12
2.2. Desarrollo . . . 12
2.2.1. Definición formal del problema . . . 12
2.2.2. Enfoques existentes y estado del arte . . . 13
2.2.3. Contexto y aplicaciones específicas . . . 13
2.2.4. Métodos utilizados . . . 13
2.3. Algoritmos . . . 13
2.3.1. Algoritmos Tabulares (Entorno Taxi-v3) . . . 13
2.3.2. Algoritmos de Aproximación (Entorno LunarLander-v3) . . . 15
2.4. Evaluación y Experimentos . . . 16
2.4.1. Configuración Experimental . . . 16
2.4.2. Resultados en Entorno Tabular: Taxi-v3 . . . 17
2.4.3. Resultados en Entorno Continuo: LunarLander-v3 . . . 18
2.4.4. Análisis Comparativo Global . . . 20
2.5. Conclusiones . . . 20
2.5.1. Resumen de los principales resultados . . . 21
2.5.2. Limitaciones del estudio y posibles mejoras . . . 21
2.5.3. Líneas futuras de estudio . . . 21

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

El lector podrá encontrar los experimentos realizados explicados y justificados en mayor detalle en los notebooks que se han desarrollado.

1.1.1. Descripción del problema y relevancia

El problema del **Bandido Multibrazo** (o *k-armed bandit*) modela una situación de toma de decisiones en presencia de incertidumbre. En este escenario, un agente debe elegir repetidas veces entre las k palancas o "brazos"de una máquina, donde cada uno ofrece recompensas que siguen una distribución de probabilidad desconocida para el agente. De esta manera, el objetivo es maximizar la recompensa acumulada a lo largo de un horizonte temporal T.

Este problema es un caso de estudio fundamental en el campo del Aprendizaje por Refuerzo, por ser un ejemplo básico del **dilema de la Exploración-Explotación**. El agente debe decidir continuamente entre explotar el brazo que cree que ofrece la mayor recompensa (basándose en su conocimiento actual) o explorar otros brazos para recabar más información, arriesgándose a obtener recompensas menores a corto plazo pero con la posibilidad de descubrir opciones óptimas a largo plazo.

1.1.2. Motivación del trabajo

El estudio del bandido de k-brazos es importante, tanto como base teórica para comprender problemas más complejos, como por su aplicabilidad en múltiples y variados problemas del mundo real.

Ejemplos de estas aplicaciones incluyen la **optimización de campañas de publicidad online** (donde se modela la tasa de clics o CTR mediante distribuciones de Bernoulli), la **personalización de recomendaciones** en plataformas de streaming, o la **asignación de promociones** en aplicaciones de compra online.

La motivación principal de este trabajo es comprender cómo diferentes algoritmos y distribuciones de recompensa afectan al rendimiento de los agentes en este entorno.

1.1.3. Objetivos del informe

El objetivo principal de este informe técnico es realizar un estudio comparativo de diferentes familias de algoritmos de aprendizaje por refuerzo aplicados al problema del bandido de k-brazos. Los objetivos específicos son:

- **Implementación de Algoritmos:** Desarrollar y validar algoritmos representativos de las familias ϵ-Greedy, Softmax y UCB1/UCB2.
- **Análisis Experimental:** Evaluar el rendimiento de dichos algoritmos en entornos con distintas distribuciones de recompensa (Normal, Bernoulli y Binomial), siempre considerando las mismas recompensas de cada brazo para garantizar una comparación justa.
- **Evaluación de Métricas:** Analizar la eficacia de los agentes mediante las métricas de Recompensa Promedio, Porcentaje de Selección del Brazo Óptimo, Estadísticas por Brazo y Regret Acumulado.
- **Estudio de Hiperparámetros:** Investigar cómo afectan parámetros como la tasa de exploración ϵ o la temperatura τ en el comportamiento de convergencia y estabilidad de los agentes.

1.1.4. Organización del documento

El resto del presente documento se estructura de la siguiente manera:

- En la **Sección 2**, se detallan los fundamentos teóricos de los algoritmos implementados (ϵ-Greedy, Softmax, UCB1), así como antecedentes relevantes en la literatura sobre el problema en cuestión.
- La **Sección 3** describe la metodología experimental, incluyendo la explicación de los algoritmos implementados.
- La **Sección 4** presenta los resultados experimentales obtenidos, para lo cual se incluyen gráficos y métricas, junto con un análisis crítico de los mismos. También se realizan comparaciones en el caso de ϵ-Greedy con ϵ-decaimiento y UCB1 con UCB2.
- Finalmente, en la **Sección 5**, se exponen las conclusiones principales derivadas del estudio.

1.2. Desarrollo

En esta sección se formaliza el problema abordado, se revisa el estado del arte y se detallan los métodos algorítmicos implementados para la resolución del dilema de exploración-explotación.

1.2.1. Definición Formal del Problema

El problema del Bandido de k-brazos se modela como un proceso de decisión secuencial de un solo estado. Formalmente,

definimos un conjunto de acciones $\mathcal{A} = \{1, \ldots, k\}$, donde cada acción a corresponde a la selección de uno de los brazos de la máquina.

En cada paso de tiempo discreto $t = 1, 2, \ldots, T$, el agente selecciona una acción $A_t \in \mathcal{A}$ y recibe una recompensa numérica R_t. Esta recompensa proviene de una distribución de probabilidad asociada al brazo seleccionado, $\mathcal{P}_a$, tal que $R_t \sim \mathcal{P}_{A_t}$. El valor esperado de la recompensa para una acción a se denota como:

$$q(a) = \mathbb{E}[R_t | A_t = a] \tag{1.1}$$

El objetivo del agente es maximizar la recompensa total acumulada a lo largo del horizonte temporal T.

1.2.2. Enfoques Existentes y Estado del Arte

La literatura del aprendizaje por refuerzo clasifica las estrategias de resolución del problema del bandido en función de cómo gestionan la incertidumbre de las estimaciones $Q_t(a) \approx q_*(a)$.

Los enfoques clásicos se dividen en:

- **Métodos Voraces (Greedy) y ϵ-Greedy:** Son los enfoques más simples, donde el agente alterna entre explotar el conocimiento actual con probabilidad $1-\epsilon$ y explorar aleatoriamente con probabilidad ϵ. Aunque efectivos, no consideran la varianza de las estimaciones.
- **Métodos de Intervalo de Confianza (UCB):** Basados en el principio de "optimismo ante la incertidumbre", estos algoritmos (como UCB1) seleccionan acciones basándose en el límite superior de un intervalo de confianza, favoreciendo acciones poco exploradas de manera determinista.
- **Métodos de Gradiente y Softmax:** A diferencia de los anteriores, que estiman valores de acción, estos métodos aprenden una preferencia numérica $H_t(a)$ para cada acción o utilizan una distribución de probabilidad de Boltzmann (Softmax) dependiente de una temperatura τ, permitiendo una selección de cada acción proporcional a su preferencia relativa.

1.2.3. Contexto y Aplicaciones Específicas

Aunque el problema se define teóricamente con distribuciones genéricas, este trabajo aborda su aplicación en contextos específicos modelados mediante distintas distribuciones de probabilidad, las cuales representan escenarios reales de toma de decisiones:

- **Distribución de Bernoulli:** Modela escenarios de éxito/fracaso binario, como la tasa de clics (CTR) en publicidad online ($R_t \in \{0, 1\}$).
- **Distribución Binomial:** Representa situaciones donde una acción se aplica a un lote de n usuarios, obteniendo un número de éxitos k (e.g., optimización de promociones en apps de *delivery*).
- **Distribución Normal:** Utilizada cuando la recompensa es una variable continua, adecuada para métricas de interacción de usuario como el tiempo de visualización en plataformas de *streaming*.

1.2.4. Métodos Utilizados

Para abordar el problema planteado, se han implementado y comparado tres familias de algoritmos, cada una con un mecanismo distinto de gestión del compromiso exploración-explotación.

Algoritmo ϵ-Greedy

Este método mantiene estimaciones $Q_t(a)$ del valor de cada brazo mediante promedios muestrales. La regla de selección de acción es:

$$A_t = \begin{cases} \text{argmax}_a \, Q_t(a) & \text{con probabilidad } 1-\epsilon \\ \text{acción aleatoria} & \text{con probabilidad } \epsilon \end{cases} \tag{1.2}$$

Se han realizado experimentos variando ϵ para analizar su impacto en la convergencia.

Algoritmo ϵ-decaimiento

Como variante del anterior, se ha implementado una versión de ϵ-Greedy con decaimiento inversamente proporcional al tiempo, lo que permite una exploración más intensa en las etapas iniciales y una mayor explotación a medida que se acumula información:

$$\epsilon_t = \text{máx}\left(\epsilon_{min}, \frac{\epsilon_0}{1 + \lambda_{decay} t}\right) \tag{1.3}$$

donde ϵ_0 es el valor inicial de exploración, λ_{decay} controla la velocidad de decaimiento y ϵ_{min} garantiza un nivel mínimo de exploración a largo plazo.

Algoritmo Softmax (Distribución de Gibbs)

Este método selecciona acciones basándose en probabilidades calculadas mediante una distribución de Boltzmann. La probabilidad de seleccionar la acción a en el tiempo t viene dada por:

$$\pi_t(a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^{k} e^{Q_t(b)/\tau}} \tag{1.4}$$

donde el parámetro de temperatura τ regula la aleatoriedad de la selección. Valores altos de τ tienden a la equiprobabilidad (exploración pura), mientras que $\tau \to 0$ tiende a una política voraz (explotación pura).

Algoritmo UCB1 (Upper Confidence Bound)

El algoritmo UCB1 selecciona acciones maximizando una cota superior de confianza, expresada como suma del valor estimado $Q_t(a)$ y un término de exploración:

$$A_t = \text{argmax}_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right] \tag{1.5}$$

donde $N_t(a)$ es el número de veces que la acción a ha sido seleccionada y c es una constante que controla el grado de exploración. Este término adicional fuerza la exploración de brazos poco visitados a medida que t aumenta.

Algoritmo UCB2

Como extensión del anterior, se ha implementado el algoritmo UCB2, el cual introduce una estructura de **épocas** para

mejorar la eficiencia de la exploración. A diferencia de UCB1, que reevalúa la decisión en cada paso t, UCB2 selecciona un brazo y lo mantiene activo durante un intervalo de tiempo:

$$\tau(r) = \lceil (1+\alpha)^r \rceil \tag{1.6}$$

donde $\alpha > 0$ es un parámetro que regula el incremento de las épocas. Esta estrategia reduce la frecuencia de cambio entre brazos y permite obtener teóricamente mejores cotas de rechazo asintótico.

1.3. Algoritmos

En este capítulo se detallan los procedimientos algorítmicos seleccionados para abordar el problema del bandido de k-brazos. Se han elegido tres estrategias representativas de las familias más importantes de aprendizaje por refuerzo en este contexto: métodos voraces con exploración aleatoria (ϵ-Greedy), métodos probabilísticos basados en gradiente de valor (Softmax) y métodos de intervalo de confianza determinista (UCB1).

1.3.1. Algoritmo ϵ-Greedy

Descripción y Justificación

El algoritmo ϵ-Greedy es la estrategia base por excelencia en el aprendizaje por refuerzo. Su elección se justifica por su simplicidad y su capacidad para establecer una línea base (*baseline*) de rendimiento. Funciona alternando entre dos comportamientos controlados por el hiperparámetro $\epsilon \in [0, 1]$:

- **Explotación:** Con probabilidad $1-\epsilon$, selecciona el brazo con el mayor valor estimado $Q_t(a)$ hasta el momento.
- **Exploración:** Con probabilidad ϵ, selecciona un brazo de forma uniformemente aleatoria, garantizando que todas las acciones tengan una probabilidad no nula de ser elegidas a largo plazo.

Pseudocódigo

Algorithm 1 Algoritmo ϵ-Greedy

Require: k (número de brazos), ϵ (probabilidad), T (horizonte)
Inicializar $Q(a) \leftarrow 0$ y $N(a) \leftarrow 0$ para todo $a \in \{1, \ldots, k\}$
for $t \leftarrow 1$ **to** T **do**
  Generar número aleatorio $\xi \in [0, 1]$
  if $\xi < \epsilon$ **then**
    Seleccionar brazo A_t aleatoriamente de $\{1, \ldots, k\}$
  else
    Seleccionar $A_t = \operatorname{argmax}_a Q(a)$
  end if
  Recibir recompensa R_t del brazo A_t
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.2. Algoritmo Softmax (Distribución de Gibbs)

Descripción y Justificación

A diferencia de ϵ-Greedy, el algoritmo Softmax asigna probabilidades de selección proporcionales al valor estimado de cada brazo. Se justifica su inclusión porque representa una estrategia más sofisticada que penaliza las acciones con un rendimiento más pobre, concentrando la exploración en las acciones que son casi tan buenas como la óptima, evitando así probar iteradamente las peores opciones, como sí hace *epsilon*-greedy. La temperatura τ regula la sensibilidad de estas probabilidades ante las diferencias en los valores Q.

Pseudocódigo

Algorithm 2 Algoritmo Softmax

Require: k, τ (temperatura), T (horizonte)
Inicializar $Q(a) \leftarrow 0$ y $N(a) \leftarrow 0$ para todo a
for $t \leftarrow 1$ **to** T **do**
  Calcular denominador $S \leftarrow \sum_{b=1}^{k} e^{Q(b)/\tau}$
  for $a \leftarrow 1$ **to** k **do**
    $\pi_t(a) \leftarrow \frac{e^{Q(a)/\tau}}{S}$
  end for
  Seleccionar brazo A_t muestreando de la distribución π_t
  Recibir recompensa R_t
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.3.3. Algoritmos UCB (Upper Confidence Bound)

Descripción y Justificación

El algoritmo UCB1 selecciona acciones maximizando una cota superior del intervalo de confianza. La inclusión del algoritmo UCB1 y su variante UCB2 en este estudio comparativo se justifica por dos factores críticos:

- **Garantías de Convergencia:** Auer et al. (2002) [?] demostraron que UCB1 garantiza una cota de rechazo acumulado (*regret*) que crece de manera logarítmica con respecto al tiempo ($O(\ln T)$). Esto asegura que el número de selecciones subóptimas se minimiza asintóticamente de forma eficiente.
- **Adaptabilidad Automática:** El algoritmo gestiona el compromiso exploración-explotación sin requerir la disminución manual de parámetros (como es el caso del decaimiento de ϵ o τ en Softmax). El término de exploración $\sqrt{\frac{\ln t}{N_t(a)}}$ crece con el tiempo global t pero decrece con las visitas locales $N_t(a)$, forzando a explorar brazos poco visitados inicialmente y a converger hacia la explotación del brazo óptimo conforme se reduce la incertidumbre.

Pseudocódigo

Algorithm 3 Algoritmo UCB1

Require: k, T (horizonte)
▷ Inicialización: probar cada brazo una vez
for $a \leftarrow 1$ **to** k **do**
  Seleccionar brazo a
  Recibir recompensa R, actualizar $Q(a)$, $N(a) \leftarrow 1$
end for
▷ Bucle principal
for $t \leftarrow k+1$ **to** T **do**
  Seleccionar brazo A_t que maximice:
    $A_t = \text{argmax}_a \left(Q(a) + \sqrt{\frac{2 \ln t}{N(a)}}\right)$
  Recibir recompensa R_t
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}(R_t - Q(A_t))$
end for

1.4. Evaluación y Experimentos

En este capítulo se detallan las pruebas realizadas para evaluar los algoritmos implementados. El objetivo es analizar cómo estos se comportan bajo distintas distribuciones de recompensa y configuraciones de hiperparámetros.

1.4.1. Configuración Experimental

Entornos de Prueba

Se han utilizado tres tipos de entornos de prueba basados en diferentes distribuciones de recompensa:

- **Distribución Normal ($\mathcal{N}(\mu, \sigma^2)$):** Simula recompensas continuas. Se configuraron $k = 10$ brazos con medias $\mu \in [1, 10]$ y desviación estándar $\sigma = 1$.
- **Distribución de Bernoulli ($\mathcal{B}(p)$):** Simula recompensas binarias (éxito/fracaso). Se utilizaron $k = 10$ brazos con probabilidades de éxito $p \in [0.1, 1.0]$.
- **Distribución Binomial ($\mathcal{B}(n, p)$):** Simula recompensas discretas agregadas (número de éxitos en n intentos). Se configuraron $k = 10$ brazos con $n = 10$ y $p \in [0.1, 1.0]$.

Herramientas y Reproducibilidad

Los experimentos se han implementado en Python utilizando *notebooks* de Jupyter alojados en Google Colab. Para asegurar la reproducibilidad de los experimentos, se ha fijado una semilla aleatoria (`seed`) constante en todas las ejecuciones y se han utilizado las mismas recompensas para cada brazo en todas las estrategias comparadas, garantizando así una comparación justa y directa entre los algoritmos.

Métricas de Desempeño

Para cuantificar la calidad de las estrategias, se han monitorizado las siguientes métricas a lo largo de un horizonte temporal de $T = 1000$ pasos (a excepción de la familia Softmax en la que se han utilizado 3000 pasos ya que presentaba una convergencia más lenta), promediando los resultados sobre 500 ejecuciones independientes (*runs*):

1. **Recompensa Promedio:** Evolución del valor medio obtenido en cada paso de tiempo.
2. **Porcentaje de Selección Óptima:** Proporción de veces que el agente selecciona el brazo con la mayor esperanza matemática (μ^*).
3. **Rechazo Acumulado (*Regret*):** La diferencia acumulada entre la recompensa óptima esperada y la obtenida por el agente.
4. **Estadísticas por Brazo:** Histograma de selecciones que permite visualizar si el agente convergió al brazo óptimo o quedó atrapado en óptimos locales.

1.4.2. Resultados: Familia ϵ-Greedy

Se evaluó el algoritmo ϵ-Greedy para valores $\epsilon \in \{0, 0.01, 0.1\}$, así como su variante con decaimiento (ϵ-decaimiento). A diferencia de Softmax, esta familia incluye una fase de exploración inicial explícita.

Para todas las distribuciones se observó que:

- $\epsilon = 0$ **(Explotación pura):** Tras la exploración inicial forzada, el agente selecciona siempre el brazo con mayor recompensa estimada. En el experimento base con distribución Normal ($\sigma = 1$), este enfoque obtuvo un rendimiento muy alto, ya que en la mayoría de las ejecuciones la estimación inicial fue suficientemente representativa como para identificar correctamente el brazo óptimo desde el principio. No obstante, su comportamiento es extremadamente sensible al ruido: cuando la estimación inicial es errónea, el algoritmo queda atrapado en un brazo subóptimo durante toda la ejecución. Hechos que quedaron evidenciados al aumentar la desviación típica a $\sigma = 10$, donde el rendimiento se desplomó debido a la alta probabilidad de obtener muestras iniciales no representativas.
- $\epsilon = 0.01$ **(Exploración baja):** Introduce una exploración moderada (1 % de las veces), lo que permite corregir posibles errores iniciales sin penalizar excesivamente la explotación. Su convergencia es más lenta que la de $\epsilon = 0.1$ en las primeras etapas, pero a largo plazo alcanza recompensas promedio muy cercanas al óptimo, ya que la probabilidad de seleccionar brazos subóptimos es muy reducida.
- $\epsilon = 0.1$ **(Exploración alta):** Aprende con mayor rapidez durante las primeras iteraciones, ya que explora con mayor frecuencia y detecta antes el brazo óptimo. Sin embargo, su recompensa promedio se estabiliza por debajo del óptimo teórico, debido a que mantiene permanentemente una probabilidad del 10 % de seleccionar brazos subóptimos.
- **ϵ-decaimiento:** Comienza con una exploración elevada $\epsilon_0 = 1.0$ y la reduce progresivamente según la función $\epsilon(t) = \frac{\epsilon_0}{1+\lambda t}$, con un mínimo $\epsilon_{min} = 0.01$. Este esquema combina una exploración intensiva inicial con una explotación a largo plazo, logrando el mejor compromiso global entre rapidez de aprendizaje y rendimiento.

La Figura 1.1 muestra la evolución de la recompensa promedio en el caso base con distribución Normal ($\sigma = 1$).

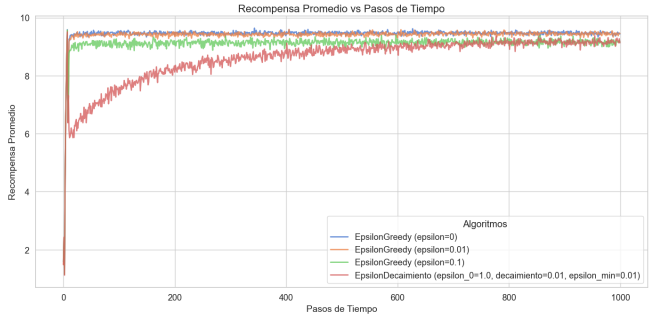

Figura 1.1: Recompensa promedio en función del número de pasos para ϵ-Greedy bajo distribución Normal con $\sigma = 1$.

Impacto de la varianza en la distribución Normal. Al aumentar la desviación típica a $\sigma = 10$, se observó una degradación significativa en el rendimiento de $\epsilon = 0$. La elevada varianza hace que las muestras iniciales sean poco representativas de la media real de cada brazo, incrementando la probabilidad de seleccionar incorrectamente un brazo subóptimo y explotarlo durante toda la ejecución.

En contraste, los algoritmos con exploración ($\epsilon = 0.01$, $\epsilon = 0.1$ y ϵ-decaimiento) logran corregir estas estimaciones erróneas gracias a la exploración continua, superando claramente al caso puramente explotador.

Por el contrario, al reducir la desviación típica a $\sigma = 0.1$, las muestras iniciales se vuelven altamente representativas. En este escenario, $\epsilon = 0$ alcanza el 100 % de selección del brazo óptimo en todas las ejecuciones, convirtiéndose en la mejor estrategia.

Este efecto puede observarse claramente en la Figura 1.2, donde se observa cómo afecta al porcentaje de selección del brazo óptimo el hecho de que la desviación típica sea $\sigma = 10$.

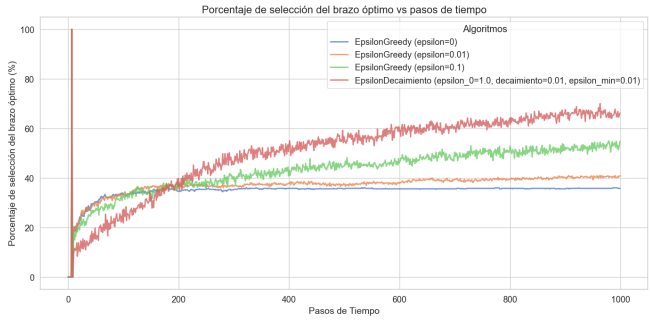

Figura 1.2: Porcentaje de selección del brazo óptimo para $\sigma = 10$ en la familia ϵ-Greedy.

Resultados en la distribución de Bernoulli. En la distribución de Bernoulli (recompensas 0 o 10), el ruido discreto introduce numerosos empates en las estimaciones iniciales y una elevada varianza en las primeras fases. Esto afecta especialmente a $\epsilon = 0$ y $\epsilon = 0.01$, que pueden quedar temporalmente atrapados en brazos subóptimos si el brazo óptimo falla en las primeras muestras, como se puede observar en la Figura 1.3.

No obstante, conforme aumenta el número de observaciones, las medias estimadas se estabilizan y todos los algoritmos mejoran progresivamente su porcentaje de selección del brazo óptimo.

En este entorno, el algoritmo ϵ-decaimiento mostró el comportamiento más robusto, ya que su fuerte exploración inicial le permite identificar correctamente el brazo óptimo incluso cuando las primeras observaciones son desfavorables.

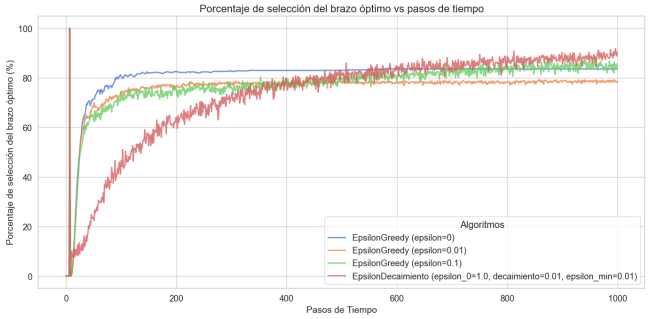

Figura 1.3: Porcentaje de selección del brazo óptimo en distribuciones Bernoulli para los algoritmos ϵ-Greedy.

Resultados en la distribución Binomial. En la distribución Binomial ($n = 10$), la varianza es menor que en la Bernoulli pura, ya que cada recompensa es la suma de varios ensayos independientes. Esto reduce los empates y estabiliza las estimaciones más rápidamente.

Como consecuencia, el comportamiento observado es muy similar al caso Normal: todos los algoritmos logran identificar el brazo óptimo con el tiempo, siendo ϵ-decaimiento y $\epsilon = 0.01$ los que presentan el mejor rendimiento global a largo plazo.

En conjunto, los resultados muestran que el desempeño de la familia ϵ-Greedy depende críticamente del equilibrio entre exploración y explotación y de la varianza de la distribución de recompensas. Una exploración constante elevada acelera el aprendizaje inicial pero penaliza el rendimiento final, mientras que la ausencia total de exploración solo es adecuada cuando las estimaciones iniciales son fiables. La estrategia con decaimiento emerge como la opción más robusta y eficiente en entornos con incertidumbre moderada o alta.

1.4.3. Resultados: Familia Softmax

Se evaluó el algoritmo Softmax variando la temperatura $\tau \in \{0.1, 1.0, 5.0\}$.

Para todas las distribuciones, se observó que:

- **Temperatura Baja ($\tau = 0.1$):** El agente se comportó de manera casi voraz, explotando rápidamente el primer brazo que visitó. Esto se debe a que Softmax no incluye una primera pasada de exploración forzada, lo que hace que la primera acción seleccionada tenga un impacto desproporcionado en las estimaciones iniciales. Esto provoca que los brazos se hayan seleccionado de manera prácticamente uniforme, como se puede observar en los histogramas de la Figura 1.4.
- **Temperatura Media ($\tau = 1.0$):** Es el que mejor ren-

dimiento mostró, logrando un equilibrio adecuado entre exploración y explotación. No obstante, el porcentaje de selección del brazo óptimo es muy inferior a los obtenidos con ϵ-Greedy o UCB1, lo que indica que el agente se quedó atrapado en óptimos locales.

- **Temperatura Alta** ($\tau = 5.0$): Al aumentar la temperatura, el agente se vuelve más explorador. No obstante, como se se observa en el histograma de la Figura 1.5, la selección de brazos se centra en el subconjunto de estos que tienen recompensas más altas, lo cual es coherente con la naturaleza del algoritmo.

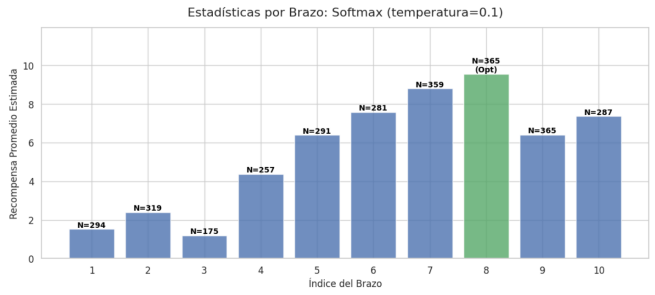

Figura 1.4: Histograma de selección de brazos con distribución Normal para el algoritmo Softmax con temperatura $tau = 0.1$.

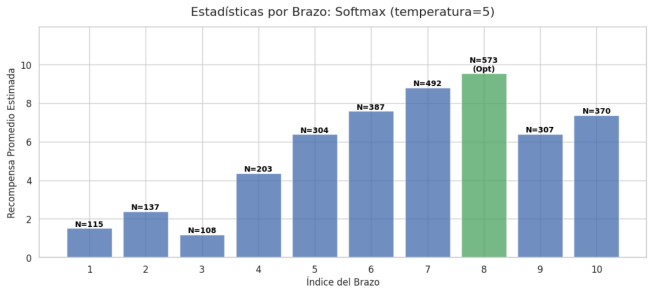

Figura 1.5: Histograma de selección de brazos con distribución Normal para el algoritmo Softmax con temperatura $tau = 5.0$.

Además, se observó que el regret acumulado crece de manera lineal para todas las distribuciones, a diferencia de los algoritmos de la familia UCB que presentan un crecimiento logarítmico. Esto lo podemos ver por ejemplo para la distribución Normal en la Figura 1.6, pero es un comportamiento que se repite para las otras distribuciones.

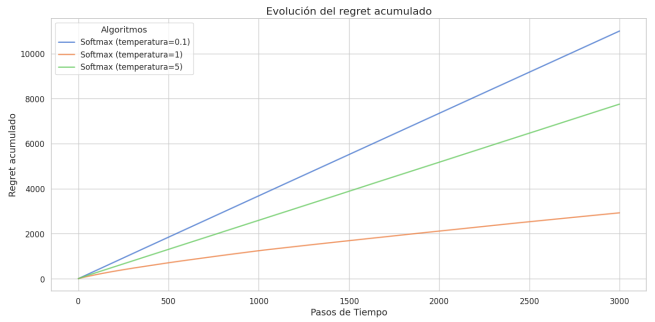

Figura 1.6: Evolución del regret acumulado para el algoritmo Softmax con distribución Normal.

Resultados distintos en la distribución de Bernoulli Al analizar los resultados en la distribución de Bernoulli , se detectó una anomalía positiva en el rendimiento de la temperatura baja ($\tau = 0.1$). A diferencia del caso Normal, donde $\tau = 0.1$ fallaba estrepitosamente, en el entorno binario su rendimiento mejoró notablemente, acercándose al de $\tau = 5.0$.

Este fenómeno se atribuye a la naturaleza discreta y penalizadora de la distribución Bernoulli (recompensas de 0 o 10).

1. Cuando el algoritmo voraz ($\tau = 0.1$) selecciona un brazo subóptimo y recibe una recompensa de 0 (fracaso), el promedio estimado de dicho brazo cae drásticamente.
2. Debido a la baja temperatura, esta caída en el valor Q reduce la probabilidad de selección del brazo a casi cero de forma inmediata.
3. Los "ceros" actúan como un mecanismo de seguridad que fuerza al algoritmo a abandonar los óptimos locales y seguir explorando, mitigando el riesgo de estancamiento típico de las políticas voraces.

Esto se puede ver en la gráfica de la Figura 1.7 donde se observa que la recompensa promedio obtenida con $\tau = 0.1$ es muy similar a la obtenida con $\tau = 5.0$:

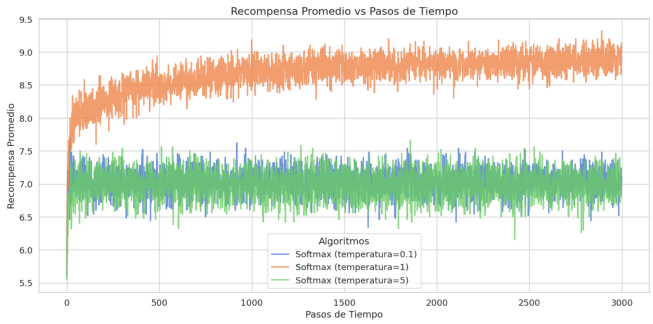

Figura 1.7: Evolución de la recompensa promedio para el algoritmo Softmax con distribución de Bernoulli.

Experimento Adicional: Impacto de la Exploración Inicial

Dado que la configuración de baja temperatura ($\tau = 0.1$) tiende a converger prematuramente a óptimos locales debido a su comportamiento voraz, se diseñó un experimento adicional para mitigar este defecto. Se introdujo una fase de **exploración inicial forzada**, obligando al agente a seleccionar cada uno de los k brazos una vez antes de aplicar la política Softmax estándar, al igual que se realiza en el algoritmo *epsilon*-greedy. Estos resultados se encuentran en la Figura 1.8.

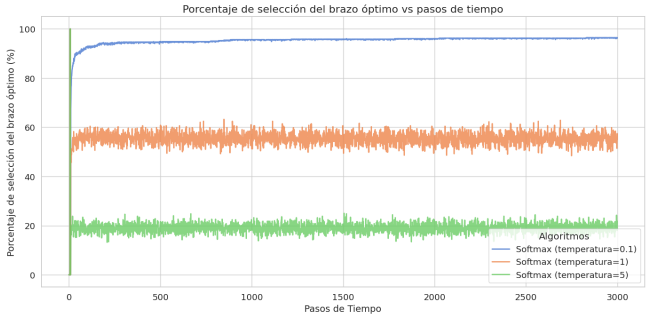

Figura 1.8: Evolución del porcentaje de selección del brazo óptimo para el algoritmo Softmax con distribución Normal con fase de exploración inicial forzada.

Resultados obtenidos:

- **Mejora Crítica en $\tau = 0.1$:** Esta primera pasada transformó radicalmente el rendimiento de esta configuración. Al garantizar una estimación inicial de todas las medias, el agente pudo identificar y explotar el brazo óptimo casi inmediatamente.
- **Aceleración de la Convergencia:** Se observó una reducción drástica en el tiempo de estabilización del algoritmo, pasando de aproximadamente 3000 pasos (en la versión estándar) a converger cerca de los 500 pasos.
- **Impacto Marginal en $\tau \geq 1.0$:** Para temperaturas medias y altas, la exploración inicial no aportó mejoras significativas, dado que la propia entropía de la distribución de Gibbs con estos valores ya garantiza una exploración suficiente por sí misma.

Este experimento confirma que las políticas casi-voraces ($\tau \to 0$) son viables y altamente eficientes si se combinan con una fase de exploración inicial que garantice una cobertura mínima de todas las acciones.

1.4.4. Resultados: Familia UCB (Upper Confidence Bound)

Se evaluaron los algoritmos UCB1 y UCB2, en el caso de UCB1 variando el parámetro de exploración $C \in \{0.5, 1, 2\}$ y en el de UCB2 variando la duración de épocas $\alpha \in \{0.1, 0.01, 0.001\}$.

Para todas las distribuciones, se observó que:

- **UCB1:** El agente presenta siempre un buen aprovechamiento de la información previamente adquirida, con una curva de aprendizaje muy pronunciada. La variación de C modifica de manera suave el equilibrio entre exploración y explotación. Con $C = 2.0$, el agente se comporta de manera más exploradora, mientras que con $C = 0.5$ se vuelve más voraz, como se puede observar en la Figura 1.9.
- **UCB2**: También es un algoritmo muy eficaz, alcanzando rendimientos similares a UCB1. Al trabajar con épocas, los agentes presentan un comportamiento oscilatorio entre explotación del brazo óptimo y exploración de brazos subóptimos, controlado por el parámetro α. El valor $\alpha = 0.1$ mostró el mejor rendimiento en nuestros experimentos, seguido por $\alpha = 0.001$. Sin embargo, para el valor intermedio de $\alpha = 0.01$ las oscilaciones mencionadas tenían un periodo de orden de magnitud similar al horizonte temporal del experimento, lo que causaba una caída considerable en el rendimiento en la mitad de cada ejecución, como se puede observar en la Figura 1.10.

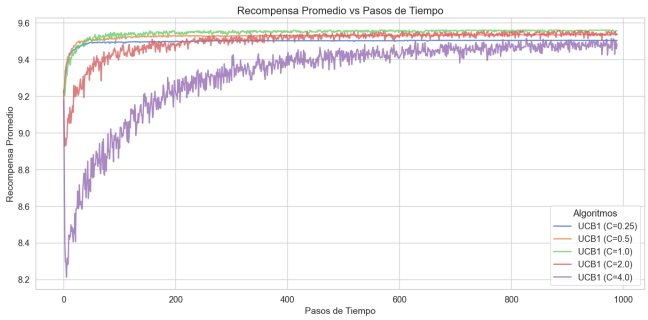

Figura 1.9: Evolución de las recompensas para el algoritmo UCB1 con distribución Normal para distintos parámetros.

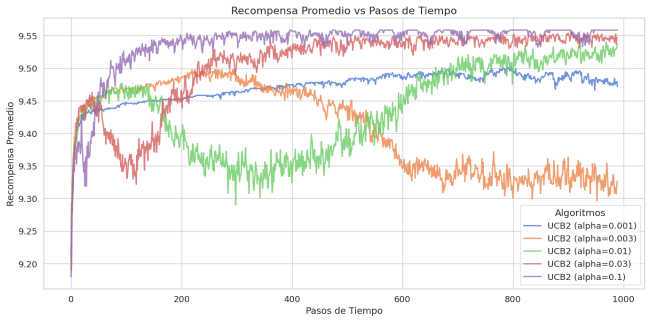

Figura 1.10: Evolución de las recompensas para el algoritmo UCB2 con distribución Normal para distintos parámetros.

También se ha comprobado empíricamente que el regret acumulado crece de manera logarítmica para ambas variantes (aunque para algunos valores de α en UCB2 no se aprecia claramente a esta escala), lo que es consistente con las garantías teóricas de estos algoritmos. Esto puede verse en la Figura 1.11 para el experimento con distribución Normal, pero es un comportamiento que se repite para las otras configuraciones.

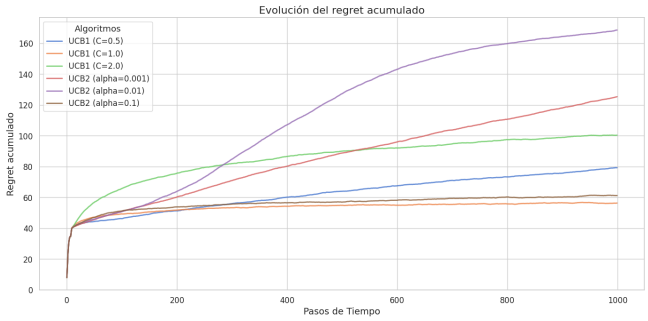

Figura 1.11: Evolución del regret acumulado para los algoritmos UCB1 y UCB2 con distribución Normal.

Comparando los resultados obtenidos entre distintos experimentos, observamos que en el experimento Binomial fueron muy similares a los obtenidos en el Normal, mientras que con

Bernoulli las recompensas acumuladas son más bajas y los regrets suben considerablemente, incluso cuadruplicándose en algunos casos, por la mayor dificultad del problema. El regret acumulado para UCB1 y UCB2 con distribución de Bernoulli se muestra en la Figura 1.12.

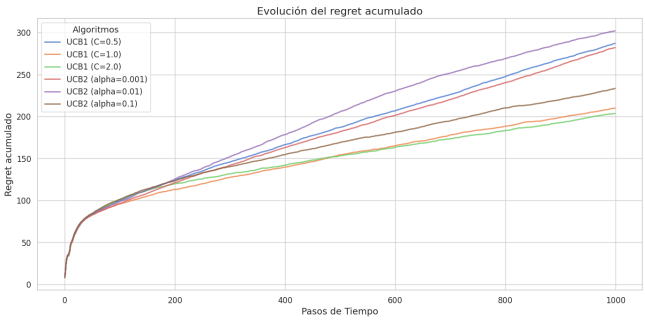

Figura 1.12: Evolución del regret acumulado para los algoritmos UCB1 y UCB2 con distribución de Bernoulli.

Resultados obtenidos:

- Ambas variantes de UCB demostraron un rendimiento superior en términos de recompensa promedio y porcentaje de selección del brazo óptimo, superando a las familias ϵ-Greedy y Softmax.
- La elección adecuada α para UCB2, es crucial para obtener un buen rendimiento con este algoritmo. A pesar de que sus cotas asintóticas son las mejores, con las duraciones de experimento que hemos usado es un algoritmo delicado y muy sensible a la configuración de este hiperparámetro, lo que puede limitar su aplicabilidad práctica.
- UCB1 mostró un rendimiento robusto y consistente en todas las configuraciones, con variaciones suaves al modificar el parámetro C.

1.4.5. Análisis Comparativo Global

Se realizó un experimento conjunto utilizando una distribución Normal con $\mu \in [1, 10]$, $\sigma = 1$, $k = 10$ brazos, 500 ejecuciones y 1000 pasos, comparando cinco algoritmos: ϵ-Greedy ($\epsilon = 0.01$), ϵ-decaimiento ($\epsilon_0 = 1$, $\lambda = 0.01$, $\epsilon_{min} = 0.01$), UCB1 ($c = 1$), UCB2 ($\alpha = 0.1$) y Softmax ($\tau = 0.1$).

Discusión de Resultados

- **Convergencia y selección óptima:** UCB1 mostró el mejor comportamiento global, alcanzando aproximadamente un 99 % de selección del brazo óptimo, seguido muy de cerca por UCB2 (98 %). Ambos métodos presentan una convergencia rápida y estable gracias a su exploración dirigida basada en incertidumbre. Softmax (95 %) y $\epsilon = 0.01$ (89 %) mantienen una exploración residual constante, lo que limita su convergencia al 100 %. A ϵ-decaimiento (91 %) le ocurre lo mismo, y además, su elevada exploración inicial disminuye su recompensa promedio al principio.
  La comparación directa del porcentaje de selección óptima se presenta en la Figura 1.13.

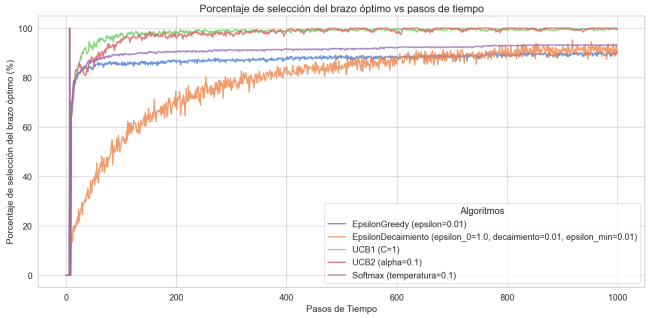

Figura 1.13: Porcentaje de selección del brazo óptimo en el experimento comparativo global.

- **Rendimiento en recompensa promedio:** Salvo ϵ-decaimiento, todos los algoritmos convergen rápidamente hacia la media del brazo óptimo (9.56 en esta instancia). UCB1, UCB2, Softmax y $\epsilon = 0.01$ alcanzan prácticamente el mismo valor final. El ϵ-decaimiento presenta un crecimiento más lento debido al coste inicial de exploración, aunque con un horizonte mayor tendería al mismo nivel.
  La evolución de la recompensa promedio puede observarse en la Figura 1.14.

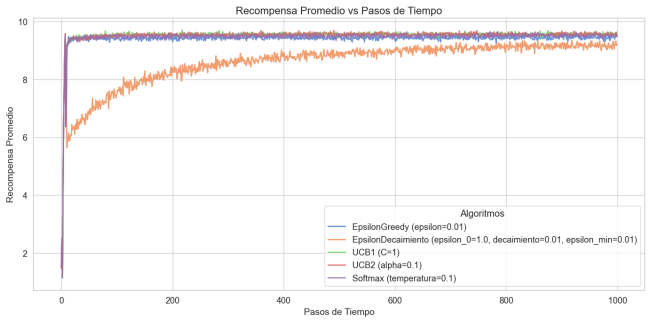

Figura 1.14: Recompensa promedio acumulada en el experimento comparativo global.

- **Regret acumulado:** UCB1 y UCB2 mantienen el menor regret global. Softmax y $\epsilon = 0.01$ presentan un crecimiento moderado debido a su exploración constante. ϵ-decaimiento acumula mayor regret en las primeras fases, lo que afecta su rendimiento observable en horizontes finitos.
  El comportamiento del regret acumulado se muestra en la Figura 1.15.

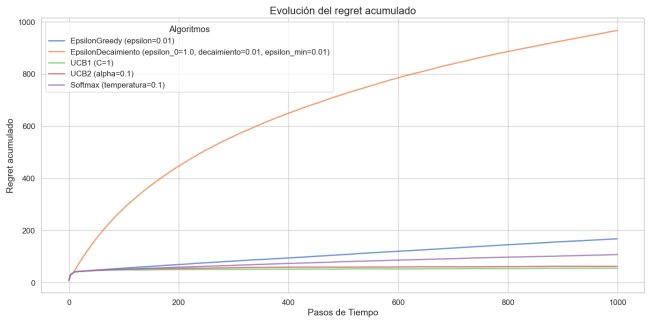

Figura 1.15: Regret acumulado en el experimento comparativo global.

- **Conclusión:** En este experimento concreto, UCB1 se posiciona como la estrategia más eficiente y robusta, combinando rápida convergencia, máxima selección óptima y bajo regret. UCB2 ofrece resultados prácticamente equivalentes, aunque algo más inestables. Softmax y ϵ-Greedy con ϵ pequeño son competitivos en recompensa promedio, pero menos eficientes en términos de identificación del brazo óptimo. El esquema con decaimiento resulta más sensible al horizonte temporal considerado, pues necesitaría como ya hemos comentado un horizonte mayor para superar su coste inicial de exploración y alcanzar el mismo rendimiento que los otros algoritmos.

1.5. Conclusiones

A partir del análisis experimental realizado sobre el problema del bandido de k-brazos bajo distintas distribuciones de recompensa, se extraen las siguientes conclusiones fundamentales:

- **Sensibilidad de los métodos básicos:** El rendimiento de la familia ϵ-Greedy depende críticamente del nivel de ruido del entorno y del ajuste de ϵ. Mientras que la explotación pura ($\epsilon = 0$) es eficiente en entornos de muy baja varianza, se obtienen muy malos resultados cuando hay mucha varianza. La introducción de un ϵ-decaimiento obtiene los resultados más robustos dentro de esta familia, aunque la gran exploración con la que empieza en los primeros episodios penaliza la recompensa en horizontes temporales cortos.
- **Dependencia paramétrica y estancamiento en Softmax:** El algoritmo Softmax es muy sensible a la selección de su temperatura τ. Temperaturas bajas ($\tau = 0.1$) provocan una convergencia prematura hacia óptimos locales debido a ser prácticamente voraz. Este defecto solo logra mitigarse de forma natural en entornos muy penalizadores (como la distribución de Bernoulli) o forzando explícitamente una pasada de exploración inicial en el algoritmo.
- **Superioridad de los métodos basados en incertidumbre (UCB):** Las estrategias UCB1 y UCB2 han demostrado ser las mejores para este problema. Al balancear automáticamente la exploración y la explotación sin depender de decaimientos manuales, logran estabilizarse rápidamente. En el análisis global comparativo, UCB1 se posicionó como la estrategia óptima, rozando un 99 % de precisión en la selección del brazo correcto y minimizando el *regret* acumulado con un crecimiento logarítmico, lo cual confirma en la práctica las garantías teóricas establecidas en la literatura.

En definitiva, se concluye que, para entornos estacionarios con incertidumbre, los algoritmos basados en la cota superior de confianza (especialmente UCB1) superan sistemáticamente a los métodos basados en probabilidad pura o exploración aleatoria constante.

Capítulo 2

Aprendizaje en Entornos Complejos: De Métodos Tabulares a Aproximaciones

2.1. Introducción

2.1.1. Descripción del problema y relevancia

En esta segunda parte, nos centramos en el estudio de entornos secuenciales, donde el entorno no es estático, sino que evoluciona en función de las acciones del agente. Estos entornos, formalizados como Procesos de Decisión de Markov (MDP), representan un desafío en el campo del Aprendizaje por Refuerzo debido a la necesidad de planificar a largo plazo y manejar espacios de estados infinitos.

Este tipo de técnicas tienen una gran relevancia debido a su aplicabilidad en una amplia gama de dominios, desde la robótica y la conducción autónoma, hasta la optimización de sistemas complejos y la toma de decisiones en finanzas.

2.1.2. Motivación del trabajo

La transición del problema del bandido de k-brazos a los Procesos de Decisión de Markov (MDP) introduce recompensas retardadas y planificación a largo plazo. Además, a medida que los problemas del mundo real se vuelven más complicados, los espacios de estados crecen de manera exponencial. Esto hace que los enfoques clásicos resulten insuficientes debido a la limitación de memoria y tiempo de cómputo (la conocida "maldición de la dimensionalidad"). Nuestra motivación radica en documentar y comprender empíricamente la transición necesaria para saltar de un entorno puramente discreto y manejable, como **Taxi-v3**, a un entorno con físicas dinámicas y espacios continuos, como **LunarLander**.

2.1.3. Objetivos del capítulo

El objetivo principal de este capítulo es implementar, evaluar y comparar diferentes algoritmos de Aprendizaje por Refuerzo capaces de manejar múltiples estados. De forma específica, buscamos:

- Desarrollar agentes basados en Q-Learning, SARSA, Montecarlo y sus variantes.
- Evaluar el rendimiento y la velocidad de convergencia de métodos de Montecarlo y Diferencias Temporales en un espacio discreto.
- Investigar el impacto de hiperparámetros como el factor de descuento γ, la tasa de aprendizaje α y el parámetro de exploración ϵ en la convergencia y el rendimiento.
- Implementar técnicas de aproximación de funciones para superar las limitaciones tabulares en espacios continuos.
- Contrastar empíricamente las propiedades *on-policy* y *off-policy* a través de distintas familias de algoritmos.

2.1.4. Organización del capítulo

El presente capítulo se estructura de la siguiente manera:

- La **Sección 1** engloba esta introducción.
- La **Sección 2** detalla el desarrollo y la formalización matemática del problema, junto con la literatura existente.
- La **Sección 3** está dedicada a los métodos tabulares aplicados al entorno **Taxi-v3**, abarcando Montecarlo y Diferencias Temporales (SARSA, Expected-SARSA, Q-Learning y Double Q-Learning). Además, explora el control con aproximaciones en el entorno **LunarLander**, donde implementamos SARSA semi-gradiente y Deep Q-Learning (DQN).
- Finalmente, la **Sección 4** presenta la evaluación de los algoritmos implementados, analizando los resultados de diversos experimentos.

2.2. Desarrollo

2.2.1. Definición formal del problema

Para extender las ideas del bandido de k-brazos a situaciones secuenciales, formalizamos el problema de toma de decisiones como un **Proceso de Decisión de Markov (MDP)**, definido por la tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$. A diferencia del capítulo anterior, en cada instante de tiempo t, el agente observa un estado $S_t \in \mathcal{S}$ y ejecuta una acción $A_t \in \mathcal{A}$. Como consecuencia, el entorno transita a un nuevo estado S_{t+1} según la probabilidad de transición $\mathcal{P}(S_{t+1}|S_t, A_t)$ y emite una recompensa $R_{t+1} \in \mathcal{R}$. Nuestro objetivo es encontrar una política $\pi(a|s)$ que maximice el retorno esperado descontado:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

donde $\gamma \in [0, 1]$ es el factor de descuento que pondera la importancia de las recompensas futuras.

2.2.2. Enfoques existentes y estado del arte

La literatura clásica de RL establece dos grandes ramas para la resolución de MDPs: los métodos tabulares y los métodos de aproximación. Para espacios discretos pequeños, algoritmos como Q-Learning y SARSA garantizan la convergencia a la política óptima construyendo un diccionario de valores. Sin embargo, para entornos continuos o de alta dimensionalidad, la literatura reciente se apoya en aproximadores de funciones no lineales. La introducción de Deep Q-Networks (DQN) por Mnih et al. marcó un punto de inflexión al demostrar que las redes neuronales profundas pueden estabilizar el aprendizaje *off-policy* mediante el uso de repetición de experiencia (*Experience Replay*) y redes objetivo (*Target Networks*).

2.2.3. Contexto y aplicaciones específicas

Para comprender la progresión de nuestro trabajo en este capítulo, es vital establecer las características de los dos entornos de prueba que utilizaremos:

- **Taxi-v3:** Un entorno discreto clásico con 500 estados posibles y 6 acciones discretas. Permite el uso de matrices $Q(s, a)$ para estimar el valor de cada par estado-acción.
- **LunarLander:** Un simulador donde el estado es un vector continuo de 8 dimensiones (coordenadas, velocidades, ángulos, etc.). Aquí, almacenar una tabla es imposible, lo que nos obliga a estimar la función de valor mediante pesos parametrizados: $\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$.

2.2.4. Métodos utilizados

Para abordar estos retos, dividimos nuestra metodología en dos bloques principales:

1. **Métodos Tabulares:** Aplicamos Montecarlo para aprender de episodios completos y Diferencias Temporales (TD) para aprender paso a paso sin esperar al final del episodio. Exploramos el control *on-policy* (SARSA, Expected-SARSA) que optimiza la política de comportamiento, y el control *off-policy* (Q-Learning, Double Q-Learning) que busca la política óptima independientemente de la exploración.
2. **Aproximaciones:** Reemplazamos la tabla Q por un modelo parametrizado. En primer lugar, utilizamos SARSA con tile coding (tilings) para construir una representación de características discreta del espacio de estados. Posteriormente, extendemos este enfoque hacia SARSA profundo, empleando redes neuronales artificiales como aproximadores de la función. Finalmente, terminamos con Deep Q-Learning (DQN), donde una red neuronal profunda aprende directamente la función Q y se encarga de la extracción automática de características.

Aprendizaje por Diferencias Temporales (TD)

A diferencia de los métodos de Montecarlo, los métodos TD permiten actualizar las estimaciones de valor basándose en otras estimaciones aprendidas, sin necesidad de esperar al final del episodio (bootstrapping). Hemos seleccionado dos enfoques fundamentales:

- **Q-Learning:** Un algoritmo *off-policy* que aprende el valor de la política óptima independientemente de las acciones tomadas por el agente.
- **Double Q-Learning:** Una variante diseñada para mitigar el sesgo de sobreestimación de los valores Q mediante el uso de dos tablas independientes.

Control con Aproximación de Funciones

Para el entorno **LunarLander**, donde el espacio de estados es continuo ($S \in \mathbb{R}^8$), se sustituye la tabla Q por un aproximador de funciones parametrizado por pesos $\mathbf{w}$.

- **SARSA Semi-gradiente con Tile Coding:** Un método *on-policy* donde la función $\hat{q}(s, a)$ se aproxima mediante una combinación lineal de características obtenidas con *tile coding*. Los pesos se actualizan mediante descenso por gradiente usando el error temporal-diferencial (TD):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$$

- **Deep SARSA:** Extensión del método anterior en la que el aproximador lineal se sustituye por una *red neuronal artificial*. La red recibe como entrada el estado y produce las estimaciones $\hat{q}(s, a, \mathbf{w})$ para cada acción. Al tratarse de un método *on-policy*, la actualización utiliza la acción realmente seleccionada por la política ϵ-greedy en el siguiente estado.
- **Deep Q-Learning (DQN):** Variante *off-policy* que aproxima la función $\hat{q}$ mediante una red neuronal profunda. A diferencia de SARSA, el objetivo de entrenamiento utiliza el valor máximo estimado en el siguiente estado:

$$y = r + \gamma \max_{a'} \hat{q}(s', a', \mathbf{w})$$

  Este enfoque permite desacoplar la política de comportamiento de la política objetivo y suele complementarse con técnicas como *experience replay* y redes objetivo para mejorar la estabilidad del aprendizaje.

2.3. Algoritmos

En este capítulo se detallan los procedimientos algorítmicos seleccionados para abordar los problemas planteados en los entornos **Taxi-v3** y **LunarLander**. Para cada algoritmo, se presenta una descripción formal, su implementación y las consideraciones técnicas relevantes.

2.3.1. Algoritmos Tabulares (Entorno Taxi-v3)

En esta sección se detallan las estrategias aplicadas al entorno discreto, donde es posible mantener una estructura de datos de tipo tabla para representar Q.

Algoritmo de Montecarlo: On-Policy y Off-Policy

A diferencia de los métodos de Diferencia Temporal, los algoritmos de Montecarlo aprenden basándose en los retornos empíricos completos de los episodios. Esto requiere matemáticamente que las tareas sean episódicas, ya que las actualizaciones de los valores se calculan de forma retrospectiva desde el estado terminal. En nuestro estudio, implementamos

y contrastamos dos variantes fundamentales para resolver el entorno: On-Policy y Off-Policy.

Montecarlo On-Policy En la variante On-Policy, evaluamos y mejoramos la misma política que utilizamos para tomar decisiones en el entorno. En nuestro experimento, empleamos una política ϵ-greedy con ϵ decaimiento lineal, hasta alcanzar un valor mínimo.

La regla de actualización para el valor de la acción A_t en el estado S_t se define como:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}\Big[G_t - Q(S_t, A_t)\Big]$$

Donde $N(S_t, A_t)$ representa el contador histórico de visitas para ese par, el cual actúa como una tasa de aprendizaje decreciente.

Pseudocódigo

Algorithm 4 Montecarlo On-Policy para Taxi-v3

Require: ϵ, γ
Inicializar $Q(s,a) \leftarrow 0$ y $N(s,a) \leftarrow 0$
for cada episodio **do**
  Generar un episodio completo usando la política ϵ-greedy basada en Q
  $G \leftarrow 0$
  for cada paso t del episodio, recorrido en orden inverso **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}[G - Q(S_t, A_t)]$
  end for
end for

Montecarlo Off-Policy En la variante Off-Policy, desacoplamos la exploración de la explotación. Nuestro objetivo es evaluar una política determinista y óptima (π), mientras interactuamos con el entorno utilizando una política de comportamiento exploratoria (b), como ϵ-greedy. Para compensar esta discrepancia en la distribución de las trayectorias, aplicamos el método de Muestreo por Importancia (Importance Sampling), ponderando los retornos empíricos según la probabilidad relativa de las acciones.

La actualización utiliza un acumulador de pesos $C(S_t, A_t)$ y un factor de importancia W:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\Big[G_t - Q(S_t, A_t)\Big]$$

Pseudocódigo

Algorithm 5 Montecarlo Off-Policy (Muestreo por Importancia) para Taxi-v3

Require: ϵ, γ
Inicializar $Q(s,a) \leftarrow 0$ y $C(s,a) \leftarrow 0$
Inicializar política objetivo $\pi(s) \leftarrow \arg\max_a Q(s,a)$
for cada episodio **do**
  Generar un episodio usando la política de comportamiento b (ϵ-greedy)
  $G \leftarrow 0$, $W \leftarrow 1$
  for cada paso t del episodio, recorrido en orden inverso **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    if $A_t \neq \pi(S_t)$ **then**
      break ▷ Divergimos entre políticas, finalizamos la actualización
    end if
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$
  end for
end for

Algoritmos SARSA y Expected-SARSA

SARSA y Expected-SARSA son algoritmos de aprendizaje por refuerzo *on-policy* basados en Diferencias Temporales, por lo que actualizan su función de valor Q tras cada acción tomada, sin necesidad de esperar al final del episodio.

SARSA En SARSA, la actualización del valor de la acción A_t en el estado S_t se basa en la acción realmente seleccionada en el siguiente estado S_{t+1}.

Pseudocódigo

Algorithm 6 SARSA para Taxi-v3

Require: α, γ, ϵ
Inicializar $Q(s,a) \leftarrow 0$
for cada episodio **do**
  $S \leftarrow$ reset entorno
  Seleccionar A desde S usando política ϵ-greedy basada en Q
  while S no es terminal **do**
    Ejecutar A, observar R, S'
    Seleccionar A' desde S' usando política ϵ-greedy basada en Q
    $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S', A') - Q(S,A)]$
    $S \leftarrow S', A \leftarrow A'$
  end while
end for

Expected-SARSA Expected-SARSA es una variante que, en lugar de usar la acción seleccionada en el siguiente estado, calcula el valor esperado sobre todas las acciones posibles, ponderado por la política ϵ-greedy. La actualización se define como:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

Algoritmo Q-Learning

Es uno de los algoritmos de aprendizaje por refuerzo más utilizados. Su uso se justifica por sus garantías teóricas de convergencia hacia la política óptima π^*, siempre que todos los pares estado-acción sean suficientemente explorados. En nuestro experimento, se utiliza una política ϵ-greedy con decaimiento lineal para equilibrar la exploración inicial del mapa del taxi con la explotación de las rutas más cortas.

La actualización del valor de la acción A_t en el estado S_t se rige por:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

Donde α es la tasa de aprendizaje y γ el factor de descuento.

Pseudocódigo

Algorithm 7 Q-Learning para Taxi-v3

Require: $\alpha, \gamma, \epsilon_{decay}$
1: Inicializar $Q(s, a)$ arbitrariamente (ej. ceros)
2: **for** cada episodio **do**
3: $S \leftarrow$ reset entorno
4: **while** S no es terminal **do**
5: Elegir A desde S usando política ϵ-greedy basada en Q
6: Ejecutar A, observar R, S'
7: $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
8: $S \leftarrow S'$
9: **end while**
10: $\epsilon \leftarrow \max(\epsilon_{min}, \epsilon - \epsilon_{decay})$
11: **end for**

Algoritmo Double Q-Learning

Q-Learning tiende a sobreestimar los valores de las acciones debido al operador máx, que puede seleccionar ruidos (positivos) en lugar de valores reales. Double Q-Learning desacopla la selección de la acción de su evaluación utilizando dos tablas, Q_1 y Q_2. Sirve para reducir la varianza en el aprendizaje y obtener una política más estable.

Con probabilidad del 50 %, actualizamos Q_1 usando Q_2 para evaluar el siguiente estado:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \\ +\alpha\left(R + \gamma Q_2(S', \operatorname{argmax}_a Q_1(S', a)) - Q_1(S, A)\right)$$

(Y viceversa para actualizar Q_2).

2.3.2. Algoritmos de Aproximación (Entorno LunarLander-v3)

Para el aterrizaje lunar, donde los sensores devuelven valores reales continuos, implementamos métodos basados en discretización de observaciones o en gradientes.

SARSA Semi-gradiente con Tile Coding

El método de tile coding es una técnica de discretización de observaciones que divide el espacio de estados en múltiples tilings superpuestos. Cada tiling es una partición del espacio de observaciones. Las observaciones continuas se representan mediante un vector de caracteristicas, y se elige la siguiente acción usando una política ϵ-greedy basada en la función de valor aproximada.

Pseudocódigo

Algorithm 8 SARSA Semi-gradiente con Tile Coding para LunarLander-v3

Require: α, γ, ϵ
1: Inicializar pesos $\mathbf{w}$ para cada acción
2: **for** cada episodio **do**
3: $S \leftarrow$ reset entorno
4: Seleccionar A desde S usando política ϵ-greedy sobre $\hat{q}(S, A) = \text{mean}(\mathbf{w}_A[\phi(S)])$ con desempate aleatorio
5: Ejecutar A, observar R, S'
6: Seleccionar A' desde S' usando política ϵ-greedy sobre $\hat{q}(S', A') = \text{mean}(\mathbf{w}_{A'}[\phi(S')])$ con desempate aleatorio
7: $\delta \leftarrow R + \gamma \text{mean}(\mathbf{w}_{A'}[\phi(S')]) - \text{mean}(\mathbf{w}_A[\phi(S)])$
8: $\mathbf{w}_A[\phi(S)] \leftarrow \mathbf{w}_A[\phi(S)] + \alpha\delta$
9: $S \leftarrow S', A \leftarrow A'$
10: **end for**

SARSA Semi-gradiente con Redes Neuronales

En este enfoque, la función de valor $Q(s, a)$ se aproxima mediante una red neuronal $\hat{q}(s, a, \mathbf{w})$. Se elige SARSA (*on-policy*) para asegurar que el agente aprenda el valor de la política que realmente está ejecutando, lo cual suele ser más estable en las etapas iniciales.

La red consta de una capa de entrada de 8 neuronas, dos capas ocultas de 128 con activación ReLU y una capa de salida de 4 neuronas. El objetivo es minimizar el error cuadrático medio (MSE) entre la salida de la red y el objetivo TD:

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}\left[\left(R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\right)^2\right]$$

La actualización de los pesos se realiza mediante el descenso de gradiente:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$$

donde δ es el error TD de SARSA.

Pseudocódigo

Algorithm 9 SARSA Semi-gradiente Deep

Require: Tasa de aprendizaje α, arquitectura $\hat{q}(\cdot, \mathbf{w})$
Inicializar pesos $\mathbf{w}$ de la red neuronal
for cada episodio **do**
  $S \leftarrow$ reset entorno
  Seleccionar A desde S usando política ϵ-greedy sobre $\hat{q}(S, \cdot, \mathbf{w})$
  while S no es terminal **do**
    Ejecutar A, observar R, S'
    Seleccionar A' desde S' usando política ϵ-greedy sobre $\hat{q}(S', \cdot, \mathbf{w})$
    Target $y \leftarrow R + \gamma \hat{q}(S', A', \mathbf{w})$
    Actualizar $\mathbf{w}$ minimizando $(y - \hat{q}(S, A, \mathbf{w}))^2$ mediante Backpropagation
    $S \leftarrow S', A \leftarrow A'$
  end while
end for

Deep Q-Learning (DQN) con Replay Buffer

A diferencia de SARSA, Deep Q-Learning es un algoritmo *off-policy* que aproxima la función de valor óptima $Q^*(s, a)$ mediante una red neuronal profunda $\hat{q}(s, a, \mathbf{w})$. Para garantizar la estabilidad del entrenamiento y evitar la divergencia matemática que suele ocurrir al usar redes neuronales en aprendizaje por refuerzo, este enfoque introduce dos mecanismos fundamentales: un *Replay Buffer* (memoria de experiencias) y una Red Objetivo (*Target Network*).

La arquitectura de nuestra red local consta de una capa de entrada correspondiente a las dimensiones del estado, dos capas ocultas de 64 neuronas con activación ReLU, y una capa de salida lineal con 4 neuronas (una para cada acción posible).

El *Replay Buffer* almacena las transiciones $(S, S', A, R, \text{done})$ a medida que el agente interactúa con el entorno. Durante el aprendizaje, tomamos mini-lotes aleatorios de esta memoria para romper la correlación temporal entre estados consecutivos. El objetivo es minimizar el error cuadrático medio (MSE) entre las predicciones de la red local y los objetivos TD calculados por la red objetivo, cuyos pesos $\mathbf{w}^-$ se actualizan de la siguiente manera:

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}_{(S,A,R,S') \sim U(D)} \left[\left(R + \gamma \max_{a'} \hat{q}(S', a', \mathbf{w}^-) \cdot (1 - \text{done}) - \hat{q}(S, A, \mathbf{w}) \right)^2 \right] \tag{2.1}$$

Siendo $U(D)$ la distribución uniforme sobre el replay buffer D.

La actualización de los pesos de la red local se realiza mediante el descenso de gradiente, y posteriormente, los pesos de la red objetivo se actualizan suavemente (*soft-update*) usando un parámetro τ:

$$\mathbf{w}^- \leftarrow \tau \mathbf{w} + (1 - \tau) \mathbf{w}^-$$

Pseudocódigo

Algorithm 10 Deep Q-Learning con Replay Buffer

Require: Tasa de aprendizaje α, factor de actualización suave τ, tamaño de lote B, frecuencia de actualización F
Inicializar pesos $\mathbf{w}$ de la red local y $\mathbf{w}^- \leftarrow \mathbf{w}$ de la red objetivo
Inicializar memoria de repetición D con capacidad máxima
$t_{step} \leftarrow 0$
for cada episodio **do**
  $S \leftarrow$ reset entorno
  while S no es terminal **do**
    Seleccionar A usando política ϵ-greedy sobre $\hat{q}(S, \cdot, \mathbf{w})$
    Ejecutar A, observar R, S', y flag de finalización *done*
    Almacenar transición $(S, S', A, R, done)$ en D
    $t_{step} \leftarrow t_{step} + 1$
    if t_{step} (mód F) $== 0$ **y** $|D| > B$ **then**
      Extraer mini-lote aleatorio de B transiciones de D
      Calcular objetivo $y \leftarrow R + \gamma \max_{a'} \hat{q}(S', a', \mathbf{w}^-) \cdot (1 - done)$
      Actualizar $\mathbf{w}$ minimizando $(y - \hat{q}(S, A, \mathbf{w}))^2$ mediante Backpropagation
      Actualizar red objetivo: $\mathbf{w}^- \leftarrow \tau \mathbf{w} + (1 - \tau) \mathbf{w}^-$
    end if
    $S \leftarrow S'$
  end while
  Actualizar ϵ según decaimiento lineal
end for

2.4. Evaluación y Experimentos

En este capítulo se detallan las pruebas realizadas para evaluar el desempeño de los algoritmos implementados en entornos de complejidad creciente: desde el espacio discreto de **Taxi-v3** hasta el espacio continuo de **LunarLander-v3**. El objetivo es analizar la convergencia, la estabilidad y el impacto de los hiperparámetros en la transición de métodos tabulares a aproximación de funciones.

2.4.1. Configuración Experimental

Entornos de Prueba

Para evaluar los algoritmos se utilizaron dos entornos clásicos de aprendizaje por refuerzo con características diferentes: uno tabular con espacio de estados discreto y otro con estados continuos que requiere aproximación funcional.

- **Taxi-v3 (Tabular).** Este entorno representa un problema de transporte en una cuadrícula donde un taxi debe recoger a un pasajero en una de las cuatro ubicaciones designadas y llevarlo a su destino. El espacio de estados contiene **500 estados discretos**, resultado de combinar las posiciones del taxi, del pasajero y del destino. El agente dispone de **6 acciones** (movimiento en las cuatro direcciones, recoger y dejar pasajero). El sistema de recompensas penaliza cada paso (-1), penaliza acciones ilegales (-10) y recompensa la entrega correcta del pasajero ($+20$). Un episodio termina cuando el pasajero es entregado o se alcanza el límite de 200 pasos.
- **LunarLander-v3 (Continuo).** Este entorno simula el aterrizaje de un módulo lunar sobre una plataforma. El estado se describe mediante un vector continuo de **8 va-**

riables, que incluyen posición, velocidad, orientación y contacto de las patas con el suelo. El agente puede realizar **4 acciones discretas** relacionadas con el control de los motores. La recompensa depende de la proximidad a la plataforma, la estabilidad del módulo y el uso de combustible, otorgando una recompensa (+100) por un aterrizaje exitoso y una fuerte penalización (−100) en caso de colisión. Debido a la naturaleza continua del espacio de estados, se emplea aproximación de la función Q mediante redes neuronales.

Herramientas y Reproducibilidad

Todo el ecosistema experimental se ha desarrollado en Python utilizando la librería *Gymnasium*. Para asegurar la reproducibilidad, se ha fijado una semilla aleatoria (`SEED = 42`) y se han utilizado envolturas (*wrappers*) para monitorizar estadísticas de entrenamiento de forma consistente.

Métricas de Desempeño

Se han monitorizado tres métricas clave promediadas en ventanas móviles de 50 a 100 episodios:

1. **Recompensa Media por Episodio:** Indica la eficacia de la política. En Taxi-v3 el objetivo es aproximarse a 8-9, mientras que en LunarLander se considera resuelto al superar los 200 puntos.
2. **Longitud del Episodio:** Se busca una reducción progresiva que indique trayectorias más directas al objetivo.
3. **Estabilidad TD (Error de entrenamiento):** En métodos tabulares representa el cambio en la tabla Q, y en aproximación, la pérdida (*loss*) de la red neuronal.

2.4.2. Resultados en Entorno Tabular: Taxi-v3

Resultados de Montecarlo

En primer lugar, se hizo una búsqueda de hiperparámetros tanto para la variante On-Policy como para la Off-Policy. Se buscó tanto el número de episodios necesarios para garantizar la convergencia, como el impacto del ratio_decay en la política ϵ-greedy. Este ratio_decay controla el momento del entrenamiento en el que se alcanza el valor mínimo de ϵ.

En la Tabla 2.1 se resumen los valores probados para cada variante:

Hiperparámetro	On-Policy	Off-Policy
Episodios (miles)	50, 150, 450	5, 10, 20, 40, 80
ratio_decay	0.5, 0.75, 1.0	0.25, 0.5, 0.75, 1.0

Tabla 2.1: Configuración de hiperparámetros para Montecarlo en Taxi-v3.

Elegimos estos valores porque Montecarlo On-Policy necesita muchos más episodios para obtener resultados aceptables.

Finalmente, los mejores parámetros encontrados fueron:

- **On-Policy:** 450.000 episodios con ratio_decay de 0.75, alcanzando una recompensa media de -21 en test.
- **Off-Policy:** 20.000 episodios con ratio_decay de 0.25, alcanzando una recompensa media de 7 en test.

Para realizar una comparación justa entre ambos métodos, vamos a utilizar el mismo número de episodios (50.000) y el mismo ratio_decay (0.25). Los resultados se muestran en la Figura 2.1.

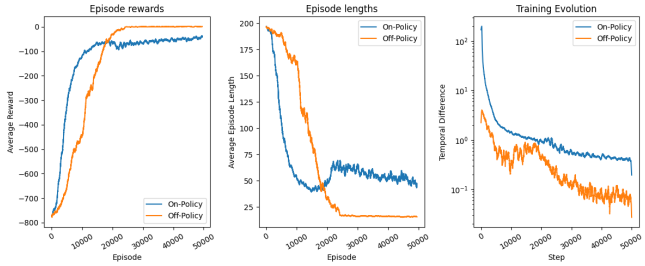

Figura 2.1: Comparativa entre Montecarlo On-Policy y Off-Policy en Taxi-v3.

Los resultados muestran que Montecarlo On-Policy no consigue superar la barrera de la recompensa 0 para este número de episodios, mientras que Off-Policy sí que consigue estabilizarse a mitad del entrenamiento.

Lo mismo ocurre con la longitud de los episodios, donde Montecarlo Off-Policy se acerca a la longitud óptima de alrededor de 13 pasos, mientras que On-Policy es mucho más inestable y solo consigue llegar a una longitud de 50 pasos.

Resultados de SARSA y Expected-SARSA

En cada uno de los dos algoritmos se han realizado los siguientes experiementos:

- Un ajuste de hiperparámetros de learning rate y número de episodios, para encontrar una buena combinación de estos. En los heatmaps de la Figura 2.2 se pueden observar los resultados obtenidos. En ellos se aprecia que, aunque ambos algoritmos alcanzan recompensas similares y muy cercanas a las óptimas en los mejores casos, Expected-SARSA muestra una mayor estabilidad en la convergencia (converge a la política óptima de manera más consistente). En el notebook `taxi_SARSA.ipynb` se encuentran resultados adicionales que apoyan esta conclusión.
- Se ha relizado un entrenamiento con los parámetros escogidos para cada algoritmo (SARSA: $\alpha = 0.24$, episodios $= 7500$; Expected-SARSA: $\alpha = 0.24$, episodios $= 4000$) y se han graficado las recompensas, longitudes de episodio y errores diferenciales. Estas gráficas se pueden ver en la Figura 2.3. En ellas se observa que ambos algoritmos mejoran conforme avanza el entrenamiento, aunque Expected-SARSA lo hace de manera más rápida. La longitud de los episodios disminuye a medida que el agente aprende a entregar el pasajero en menos pasos. El error TD muestra que la función de valor se va estabilizando de cara al final del entrenamiento, aunque sin estabilizarse completamente. [3]

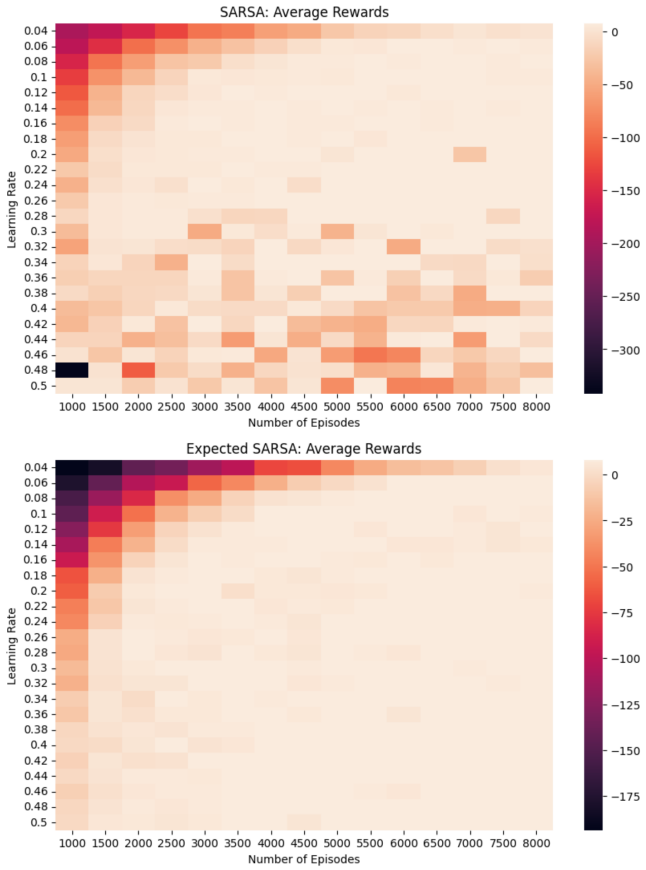

Figura 2.2: Heatmaps de recompensa media para diferentes combinaciones de learning rate y número de episodios en SARSA y Expected-SARSA.

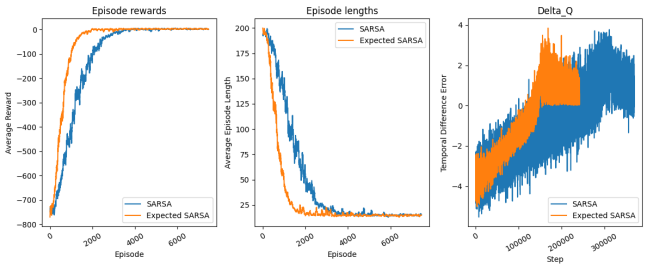

Figura 2.3: Evolución de recompensa media, longitud de episodios y error TD durante el entrenamiento de SARSA y Expected-SARSA.

Resultados de Q-learning

Se compararon los algoritmos Q-Learning y Double Q-Learning bajo una política ϵ-greedy con decaimiento lineal.

Optimización de Hiperparámetros (Grid Search). Tras evaluar tasas de aprendizaje $\alpha \in [0.016, 1]$ y horizontes de hasta 7000 episodios, se determinó que:

- Un $\alpha = 0.256$ ofrece el mejor equilibrio entre velocidad y estabilidad. Valores cercanos a 1 provocan inestabilidad.
- El factor de descuento óptimo es $\gamma = 0.95$. Valores inferiores impiden que el agente valore correctamente la recompensa final.

Q-Learning vs. Double Q-Learning. Aunque teóricamente Double Q-Learning mitiga el sesgo de sobreestimación, en este entorno discreto los resultados mostraron matices interesantes. Como se puede apreciar en la comparativa de la Figura 2.4, Q-Learning convergió más rápido (aprox. 750 episodios) que Double Q-Learning, que necesitó más episodios (debido a la división de actualizaciones entre dos tablas) para estabilizarse.

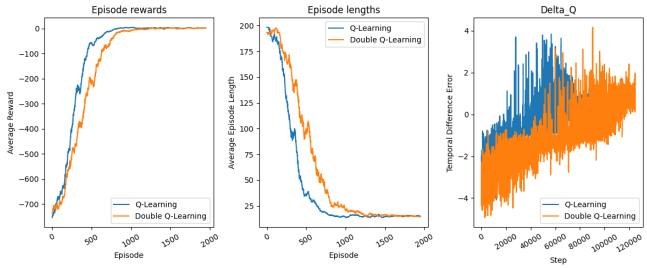

Figura 2.4: Comparación de recompensa acumulada y convergencia entre Q-Learning y Double Q-Learning en Taxi-v3.

La Figura 2.4 muestra la evolución de la recompensa promedio durante el entrenamiento. Se observa que, tras una fase inicial con recompensas muy negativas debido a la exploración aleatoria, el agente mejora progresivamente hasta estabilizarse alrededor de los 750 episodios. En paralelo, la longitud de los episodios disminuye, indicando que el agente aprende a completar la tarea en menos pasos. La estabilización del error TD confirma que las estimaciones de la función Q han convergido hacia una política estable.

En fase de test, ambos lograron un *Win Rate* superior al 99.5 %. La Figura 2.5 ilustra la política final aprendida por el agente Q-Learning, donde se observa cómo el taxi ha optimizado las rutas de recogida y entrega en los puntos clave del mapa.

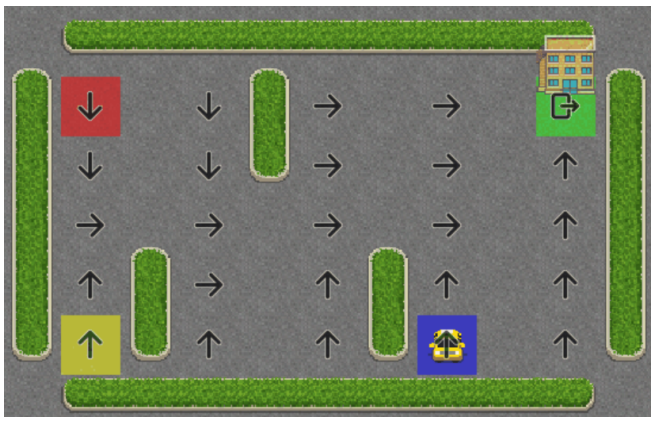

Figura 2.5: Representación visual de la política óptima para el agente Q-Learning en el entorno Taxi-v3.

2.4.3. Resultados en Entorno Continuo: LunarLander-v3

Resultados de SARSA Semi-gradiente con tile coding

Para evaluar el algoritmo SARSA Semi-gradiente con tile coding para la representación de los estados, comenzamos realizando una búsqueda de hiperparámetros para encontrar la mejor combinación de número de episodios frente al número de subdivisiones por dimensión continua. En la Figura 2.6 se pueden observar los resultados obtenidos, donde se aprecia que todas las combinaciones dan muy malos resultados. Tras seleccionar la mejor combinación (a sabiendas de que la combinación ganadora es probablemente fruto de la aleatoriedad, más que de los valores de los hiperparámetros), se realizó un entrenamiento con esta configuración y se graficaron las métricas de recompensa, longitud de episodios y error TD. Estas gráficas se pueden ver en la Figura 2.7. En ella se observa que la recompensa no solo no mejora con el avance del entrenamiento, sino que disminuye. Creemos que esto se debe a la alta dimensionalidad de las observaciones del problema. Para tener grano fino en la representación de los estados, es necesario un número elevado de subdivisiones por dimensión, y el número de tiles crece exponencialmente con las subdivisiones. Esto hace que de cara al final del entrenamiento, el agente no haya visitado suficientes estados para aprender una política coherente.

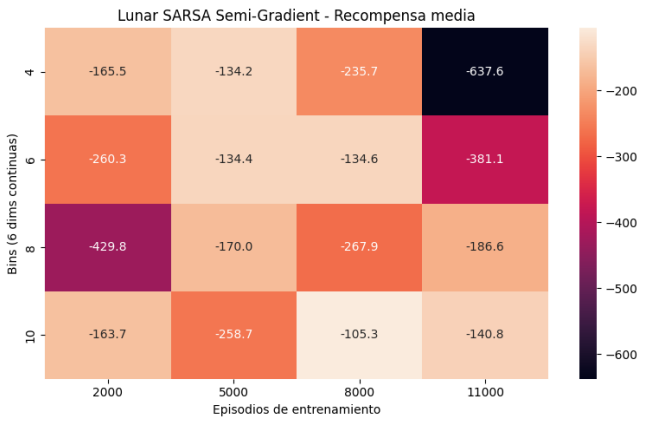

Figura 2.6: Heatmap de recompensa media para diferentes combinaciones de número de episodios y número de subdivisiones por dimensión en SARSA Semi-gradiente con tile coding.

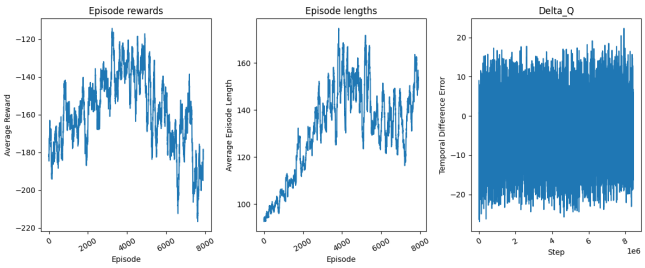

Figura 2.7: Evolución de recompensa media, longitud de episodios y error TD durante el entrenamiento de SARSA Semi-gradiente con tile coding.

Resultados de SARSA Semi-gradiente con red neuronal

En este entorno, se evaluó el algoritmo SARSA Semi-gradiente utilizando una arquitectura de red neuronal profunda (DNN).

Impacto de la Exploración y el Aprendizaje.

- **ϵ-Greedy vs. ϵ-Decay:** El uso de un decaimiento progresivo ($\epsilon_0 = 1.0 \rightarrow \epsilon_f = 0.05$) elevó el *Win Rate* del 78.9 % al 89.7 %. Esto confirma que en espacios continuos, una exploración inicial exhaustiva es fundamental para "mapear" la física del entorno. En la Figura 2.8 se detallan las métricas de este experimento
- **Sensibilidad a α**: A diferencia del caso tabular, LunarLander mostró un comportamiento sensible a tasas de aprendizaje altas. Un $\alpha = 0.01$ dificultó enormemente la convergencia (*Win Rate* 5.3 %), mientras que $\alpha = 0.001$ permitió un aprendizaje suave y estable.

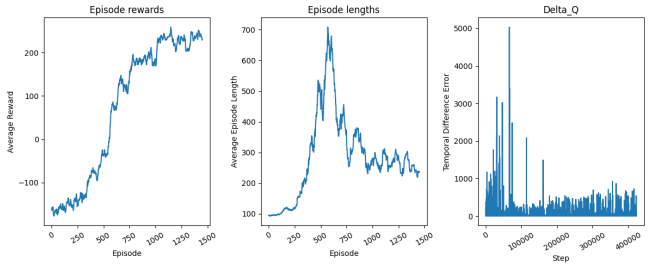

Figura 2.8: Métricas de entrenamiento (Recompensa, Longitud y Error TD) bajo la política de decaimiento lineal en LunarLander-v3 con Deep-SARSA.

En la Figura 2.8 se puede observar la evolución de las métricas principales durante el entrenamiento. A partir de aproximadamente 600 episodios, la recompensa media comienza a acercarse a valores próximos a 200, lo que indica que el agente empieza a realizar aterrizajes exitosos de forma consistente. La longitud de los episodios refleja también este aprendizaje: inicialmente la nave se estrella rápidamente, pero conforme el agente aprende a estabilizar el descenso, los episodios se alargan, y posteriormente vuelven a disminuir para optimizar el uso de combustible. El error TD presenta un pico pronunciado al inicio del entrenamiento (caracterísitco de una primera fase de aprendizaje) y posteriormente evoluciona hacia valores más estables, señal de que el proceso de aprendizaje converge.

Arquitectura de la Red Neuronal. Se experimentó con capas ocultas de 32, 64 y 128 neuronas. Aunque incluso las redes pequeñas lograron aterrizajes exitosos, la configuración de **128 neuronas** obtuvo la menor varianza y la política más robusta, demostrando que la complejidad del aterrizaje requiere una capacidad de representación suficiente para capturar las interacciones entre las 8 variables de estado. Un ejemplo visual del aterrizaje existoso de este modelo se muestra en la Figura 2.9.

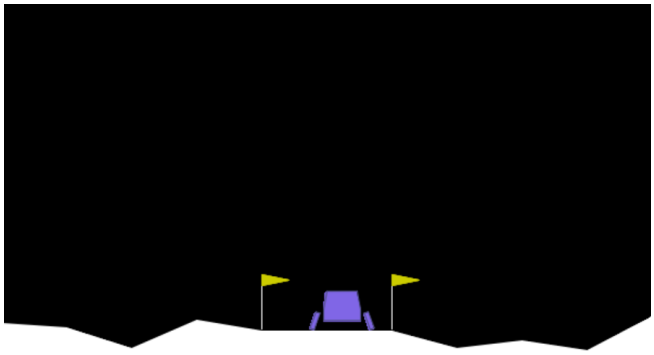

Figura 2.9: Fotograma de un aterrizaje exitoso realizado por el agente SARSA profundo con red de 128 neuronas.

Resultados de Deep Q-learning

Probamos el algoritmo DQN con la misma arquitectura de red que en SARSA profundo, con tamaño de capa oculta de 64 nueronas.

Volvemos a probar tanto ϵ-greedy como ϵ-decay y obtenemos que no hay una diferencia significativa entre ambos, pues obtienen *Win Rates* similares (93.1 % vs 96.7 %).

Podemos ver en la Figura 2.10 la evolución de las métricas durante el entrenamiento para DQN con ϵ-decay, donde se observa una tendencia similar a la de SARSA profundo, aunque con una convergencia más rápida como se verá en la sección de análisis comparativo.

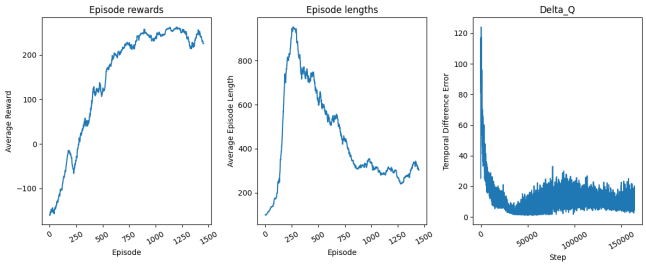

Figura 2.10: Métricas de entrenamiento (Recompensa, Longitud y Error TD) bajo la política de decaimiento lineal en LunarLander-v3 con DQN.

Por último queremos ver la influencia del parámetro de factor de descuento en este problema, para lo cual probamos con los valores de $\gamma = 0.9, 0.99, 0.999$. Nos basamos en el trabajo de [11].

Los resultados se encuentran en la Figura 2.11, donde se observa que un γ demasiado bajo (0.9) dificulta la convergencia (pues se disminuye el horizonte de información del que dispone el agente), mientras que un γ de 0.99 ofrece el mejor rendimiento, llegando a alcanzar 250 puntos de recompensa media en los últimos episodios.

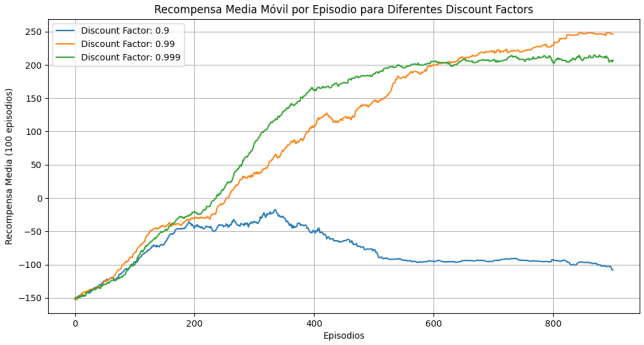

Figura 2.11: Comparativa de recompensa media durante el entrenamiento para diferentes valores de γ en DQN aplicado a LunarLander-v3.

2.4.4. Análisis Comparativo Global

Tras evaluar los tres enfoques en el entorno continuo *LunarLander-v3*, comparamos su rendimiento en la Figura 2.12, donde se observa claramente la superioridad de DQN frente a SARSA profundo, y el fracaso total de tile coding.

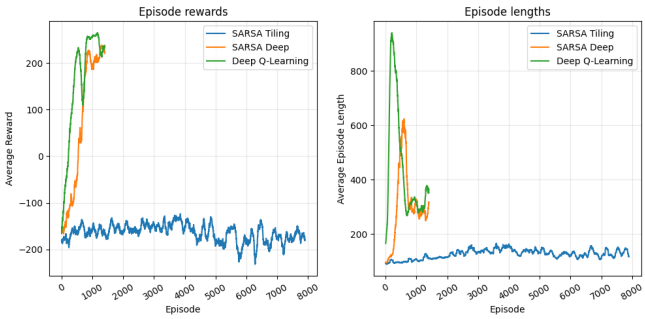

Figura 2.12: Comparativa de recompensa media durante el entrenamiento entre SARSA con tile coding, SARSA deep y DQN en LunarLander-v3.

- El algoritmo SARSA utilizando *Tile Coding* fue incapaz de resolver el entorno, obteniendo un *Win Rate* del 0.0 % y recompensas muy negativas. Esto se debe a la alta dimensionalidad que presenta este entorno, con 6 variables continuas.
- Al sustituir la discretización por una Red Neuronal Artificial con 128 neuronas en sus capas ocultas, logramos un gran avance, alcanzando un *Win Rate* cercano al 90 %.
- DQN ha sido el algoritmo más robusto y eficiente para este problema de control continuo. Logra la convergencia en etapas más tempranas y alcanza puntuaciones medias superiores (estabilizándose en torno a los 250 puntos).
- **Sensibilidad paramétrica:** En las arquitecturas observamos una fuerte dependencia de los hiperparámetros. Un factor de descuento bajo ($\gamma = 0.9$) vuelve al agente miope, impidiéndole asociar sus maniobras de estabilización con la recompensa del aterrizaje final (+100).

2.5. Conclusiones

A lo largo de este trabajo hemos documentado y contrastado empíricamente la transición desde entornos tabulares discretos hasta espacios continuos que requieren aproxima-

ción de funciones. A continuación, presentamos las conclusiones extraídas tras analizar los resultados en **Taxi-v3** y **LunarLander-v3**.

2.5.1. Resumen de los principales resultados

En primer lugar, la evaluación en el entorno discreto **Taxi-v3** nos ha permitido observar el comportamiento de los métodos clásicos. Comprobamos que las variantes *off-policy* tienden a ser más estables y eficientes. Además, *Expected-SARSA* superó a *SARSA* al reducir la varianza en sus actualizaciones. Como cabía esperar, *Q-Learning* convergió más rápido que *Double Q-Learning*, lo que nos indica que, en entornos discretos simples, el coste de dividir las actualizaciones en dos tablas no siempre compensa la mitigación del sesgo de sobreestimación.

Al dar el salto al entorno continuo **LunarLander-v3**, nos topamos de frente con la conocida "maldición de la dimensionalidad". Nuestro intento de resolverlo utilizando *SARSA con Tile Coding* fracasó por completo (0.0 % de *Win Rate*), demostrando que la discretización manual de un espacio de 8 dimensiones impide al agente visitar suficientes estados para aprender una política útil.

La solución pasó necesariamente por el Aprendizaje Profundo. Al integrar una red neuronal, *Deep SARSA* (*on-policy*) logró resolver el entorno, pero fue *Deep Q-Learning (DQN)* (*off-policy*) el algoritmo que demostró una superioridad indiscutible. La convergencia más rápida y estable de DQN se justifica por dos mecanismos que rompen con las limitaciones tabulares: el uso del *Experience Replay*, que elimina la fuerte correlación temporal de las observaciones, y la *Target Network*, que estabiliza el cálculo de los gradientes evitando oscilaciones.

Finalmente, a lo largo de todos los experimentos, comprobamos empíricamente la alta sensibilidad de estos modelos a los hiperparámetros. Vimos cómo un factor de descuento bajo ($\gamma = 0.9$) vuelve miope al agente, impidiéndole planificar el aterrizaje final, y cómo una exploración inicial estructurada (ϵ-decaimiento) resulta necesaria para mapear correctamente las dinámicas del entorno.

2.5.2. Limitaciones del estudio y posibles mejoras

A pesar de haber logrado resolver los entornos planteados, nuestro estudio presenta ciertas limitaciones. En primer lugar, aunque **LunarLander-v3** tiene un espacio de estados continuo, las acciones que el agente puede tomar siguen siendo discretas. Las arquitecturas basadas en valores Q como DQN no escalan de forma natural a espacios de acción continuos, lo que limitaría su uso directo en problemas reales de robótica de precisión.

Por otro lado, la exploración del espacio de hiperparámetros (tasas de aprendizaje, arquitecturas de red, decaimientos) la realizamos de manera mayoritariamente heurística debido a las limitaciones de capacidad de cómputo, por lo que es posible que las configuraciones utilizadas no sean el óptimo global para estos algoritmos.

2.5.3. Líneas futuras de estudio

Partiendo de la base empírica establecida en este trabajo, proponemos dos vías principales para continuar y expandir la investigación:

- **Mitigación del sesgo con Double Deep Q-Learning (DDQN):** Implementar esta extensión directa de nuestra arquitectura actual. Puesto que el algoritmo DQN estándar tiende a sobreestimar sistemáticamente los valores de las acciones al utilizar la misma red para seleccionar y evaluar, aplicar DDQN nos permitiría desacoplar estas decisiones. Sería de gran interés cuantificar si esta reducción del sesgo acelera y estabiliza aún más la convergencia en el aterrizaje lunar.
- **Transición a arquitecturas de Actor-Crítico:** Explorar algoritmos que combinan el gradiente de política con la estimación de valor. A diferencia de enfoques como DQN, que están limitados estructuralmente a espacios de acción discretos, el paradigma Actor-Crítico nos permitiría operar de forma nativa con acciones continuas. Esto abriría la puerta a controlar los motores de la nave aplicando niveles de empuje graduales en lugar de comandos binarios, acercando la simulación a un escenario de control robótico real.

Bibliografía

[1] Ginés Carrillo, Yago Ibarrola, and Aarón Palomar. Eml_carrillo_ibarrola_palomar. Repositorio de GitHub, 2025.

[2] Google. Gemini 3.1 pro. LLM, 2026.

[3] Luis Daniel Hernández Molinero. Aprendizaje por refuerzo: Bandido multibrazo. Diapositivas de clase, 2025. Máster en Inteligencia Artificial, Universidad de Murcia.

[4] Luis Daniel Hernández Molinero. Aprendizaje por refuerzo: Control on policy con aproximaciones. Diapositivas de clase, 2025. Máster en Inteligencia Artificial, Universidad de Murcia.

[5] Luis Daniel Hernández Molinero. Aprendizaje por refuerzo: Métodos de diferencias temporales. Diapositivas de clase, 2025. Máster en Inteligencia Artificial, Universidad de Murcia.

[6] Luis Daniel Hernández Molinero. Aprendizaje por refuerzo: Métodos de monte carlo. Diapositivas de clase, 2025. Máster en Inteligencia Artificial, Universidad de Murcia.

[7] Luis Daniel Hernández Molinero. Aprendizaje por refuerzo: Predicción on policy con aproximaciones. Diapositivas de clase, 2025. Máster en Inteligencia Artificial, Universidad de Murcia.

[8] Luis Daniel Hernández Molinero. eml_approximate. Repositorio de GitHub, 2025.

[9] Luis Daniel Hernández Molinero. eml_k_bandit. Repositorio de GitHub, 2025.

[10] Luis Daniel Hernández Molinero. eml_tabular. Repositorio de GitHub, 2025.

[11] Le Hoang Van. Solving lunar lander with deep q-network. *Proyecto independiente / Informe de estudiante*, 2019.